# Lecture 2

# Processors and Assembly Code

Mingyu Gao

gaomy@tsinghua.edu.cn

*IIIS, Tsinghua*

# Today's Outline

- High-level models of computers and processors
- Hardware structures of processors
  - Simple, pipelining, superscalar
  - Multi-core and GPU
  - Specialized hardware
- ISA and assembly code
  - System states
  - Arithmetic/logic, load/store, branch/jump instructions
  - Support for parallelism: SIMD and atomic instructions
  - Memory layout of a program: stack management and procedure calls

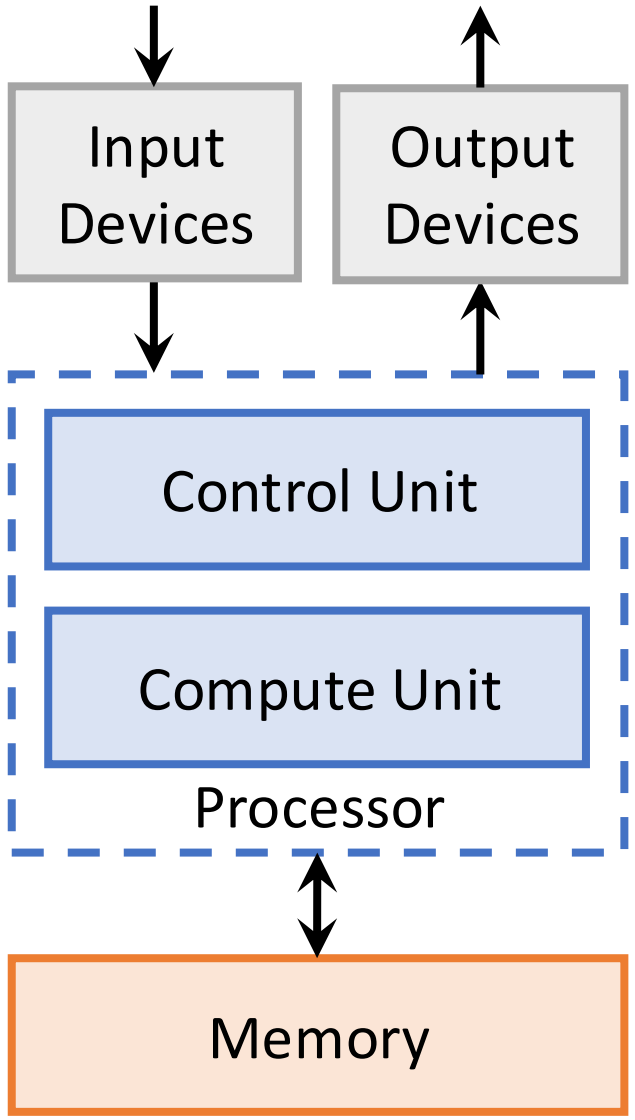
# The Simplest Model of Computer Systems

- What does a computer system do?
  1. Take in some input data from external, e.g., human, network, disks/SSDs, …
  2. Process the input data to generate results as the output data
  3. Send the output data to external, e.g., human, network, disks/SSDs, …

- What does a computer system need?
  - Input/output devices, for communicating with external world
  - Compute unit, for data processing
  - Memory, for data storage
  - Control unit, for allowing various processing functions

# Von Neumann (a.k.a., Princeton) Architecture

- Components
  - Compute unit: include arithmetic/logic units and registers
  - Memory: for both data and programs
  - Control unit: follow user program flow
  - Input/output devices
- Key point: separate processor and memory
  - Data processing vs. data storage
  - Operations vs. states
- Von Neumann vs. Harvard architecture
  - Von Neumann: a single piece of memory
  - Harvard: separate data and program memories

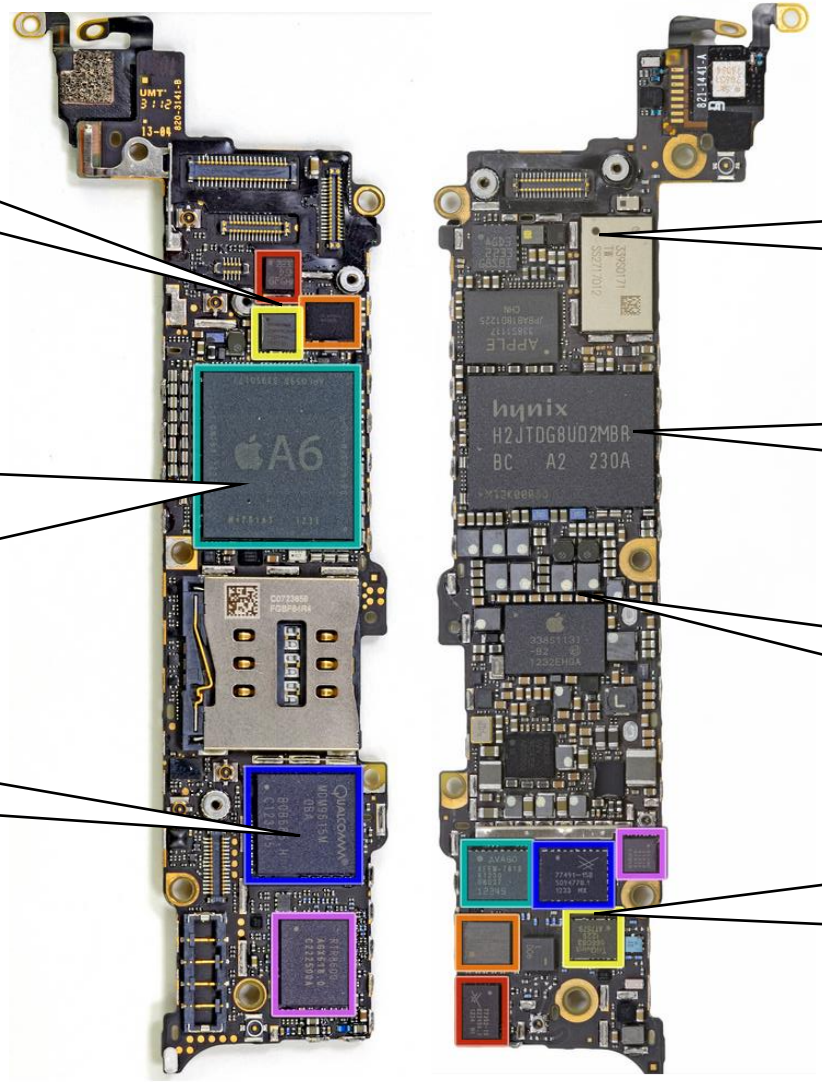


# Example: iPhone 5s Deconstructed

I/O: touch screen, accelerometer

Processor + Memory: 2 ARM cores, GPU, image/video processor; in package DRAM

Customized HW: Qualcomm LTE modem



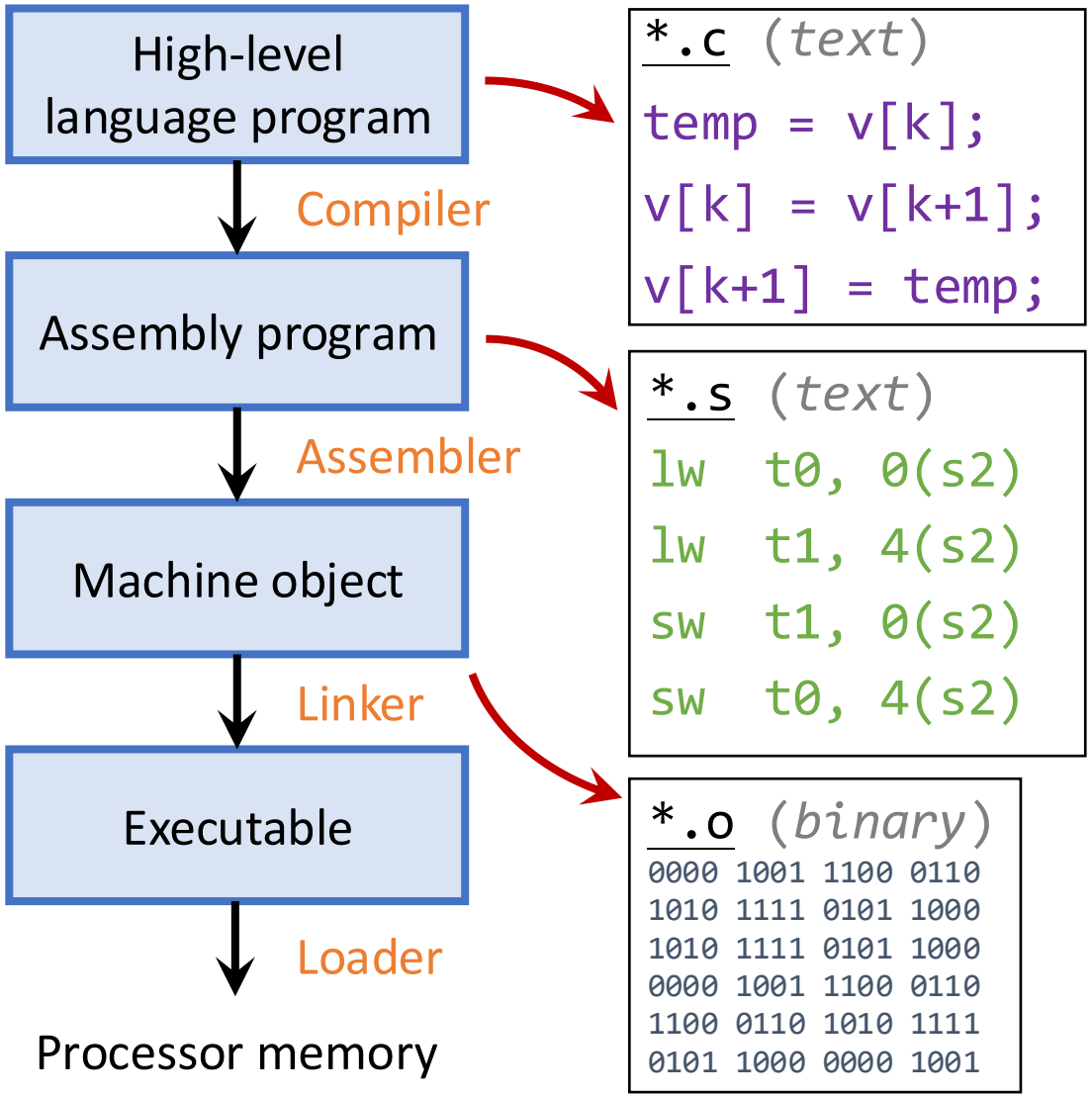

I/O: audio, WiFi, gyroscope

Memory: Hynix Flash

Customized HW: Power management

I/O + Customized HW: antenna, amplifier, duplexer

# Review: Code Compilation and Translation




- A program is compiled into a sequence of instructions
  - Instructions are encoded as bits
- Machine code: the binary sequence that encodes program instructions
  - Actual 0's and 1's stored in the memory
- Assembly code: the text representation of machine code
  - Human-readable instruction sequence

# Hardware-Software Interface: ISA

- To let a computer system do a specific job
  - A programmer writes a software program containing a sequence of instructions
  - The hardware processor follows these instructions to complete the job
- Hardware-software interface: ISA (instruction set architecture)

- Generally speaking, two types of hardware processors
  - General-purpose/programmable processors: expose an expressive and general set of instructions (i.e., Turing complete), which can realize any functionalities
    - A.k.a., CPUs, cores
    - Examples: x86, x86-64, ARM, RISC-V, MIPS, Power, IA64, …
  - Specialized/customized accelerators: expose only a limited set of commands for specific functionalities for one or a few applications
    - Examples: AI accelerators, video codec, cryptographic units, …
    - You do not really program them; you directly call these high-level commands

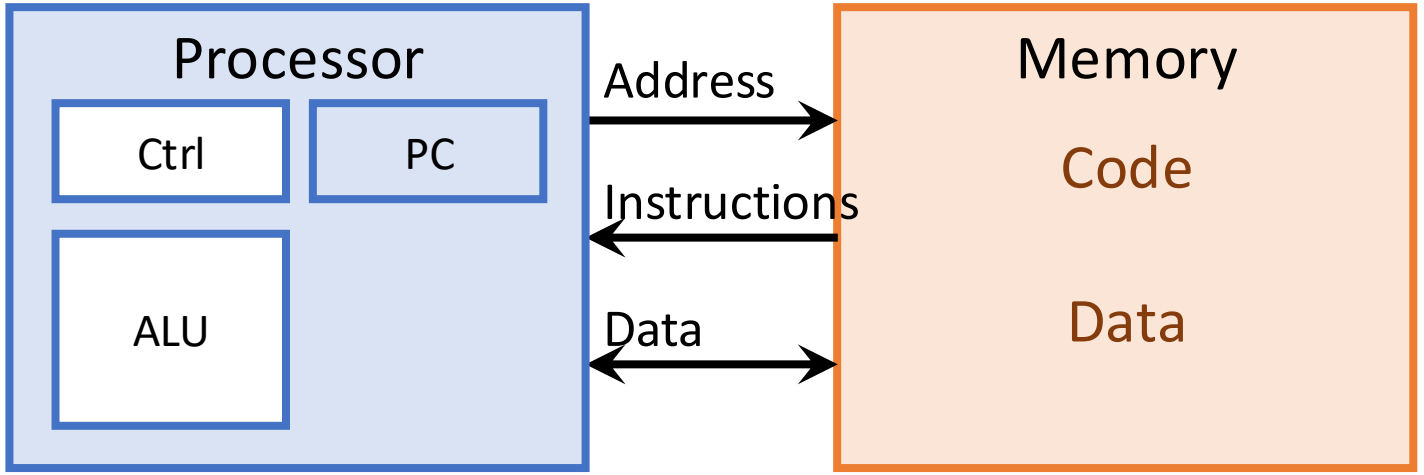
# ISA of Programmable Processors

- In this course we mainly focus on programmable processors
- ISA defines
  - The states of the processor, e.g., memory, registers, stack
  - The instructions (i.e., operations), including their functionalities and formats
- Idea of abstraction: ISA exposes functionality (architecture) but hides implementation (microarchitecture); greatly simplifies programming

- Next we first show a functionally complete but unoptimized design
- Then we add a performance optimization
  - There are many more such performance optimizations (at architectural or microarchitectural levels) in real-world processors, which make them run faster but also add complexity

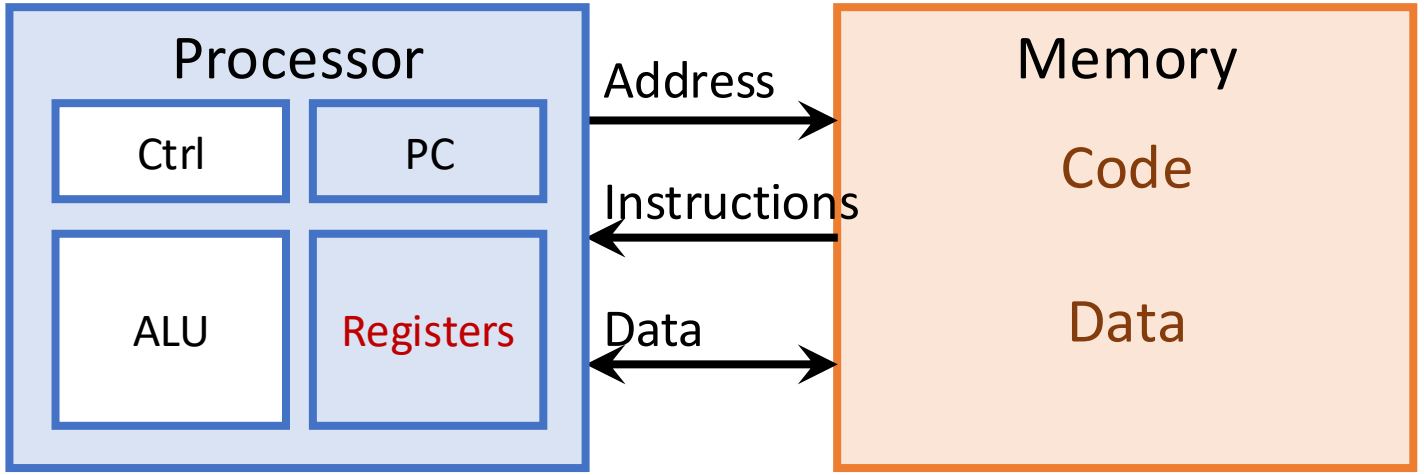
# Less Simpler Model of Processors

- State: PC, memory
  - Program counter (PC): address of the next instruction
    - Where we are in the middle of the instruction sequence
  - Memory: code and data, both stored as blocks of bytes
- Arithmetic/logic unit (ALU): digital logic to do computation
- Controller: parse instructions and decide next PC
- A problem: all data must be read/written from/to memory → too slow!



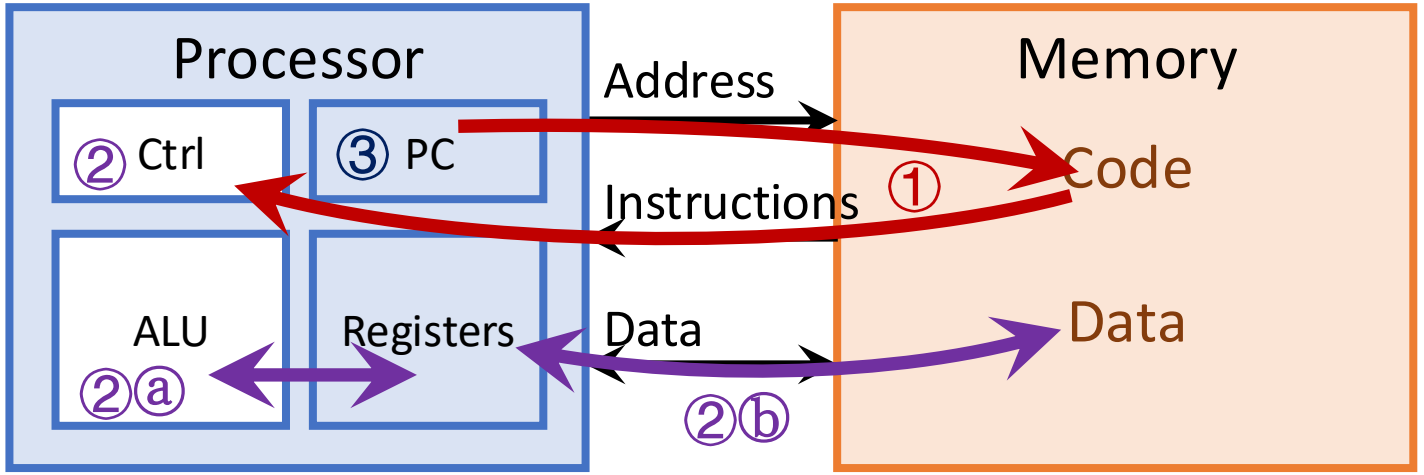
# An Optimization: Using Registers

- State: PC, memory, registers
  - Registers (a.k.a., register file): local store of heavily used data in the processor
- Smaller is faster: accessing registers is faster than accessing memory
  - More on this in the next lecture
- But only finite number of registers
  - Keep the most heavily used data in registers; explicitly managed by instructions, i.e., assembly programmers or compilers decide how to use registers



# Instruction Execution on Processors

1) PC used as address to fetch an instruction from memory
2) The controller decodes the instruction and determines what to do
   a) Compute on register values and store results to registers
   b) Transfer data between memory and processor registers
   c) …
3) Update PC to the next instruction, go back to 1)



# Today's Outline

- High-level models of computers and processors
- Hardware structures of processors
  - Simple, pipelining, superscalar
  - Multi-core and GPU
  - Specialized hardware
- ISA and assembly code
  - System states
  - Arithmetic/logic, load/store, branch/jump instructions
  - Support for parallelism: SIMD and atomic instructions
  - Memory layout of a program: stack management and procedure calls

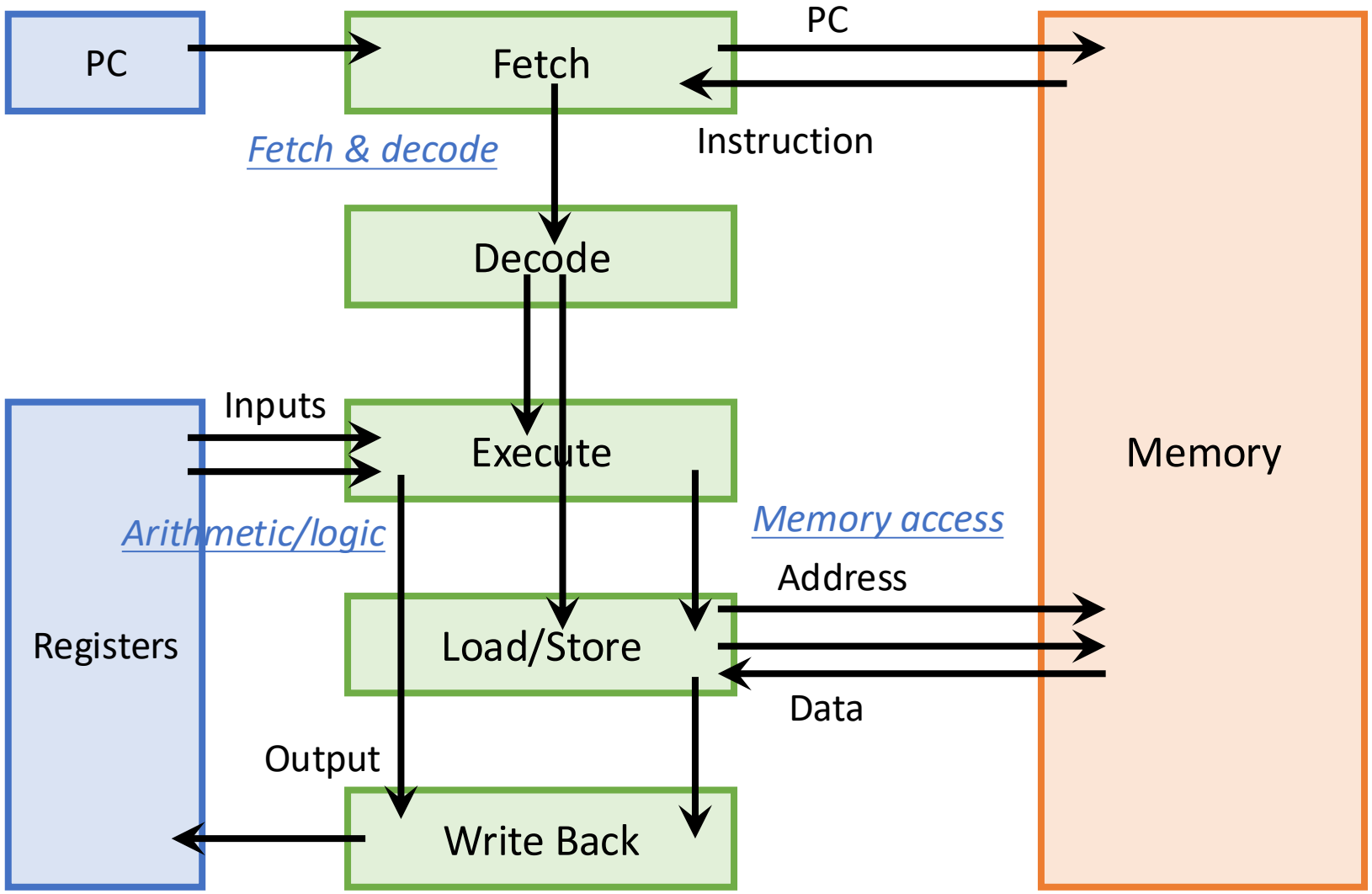
# Instruction Stages for Hardware Modules

- Break instruction execution into stages, and implement them into hardware modules, i.e., digital logic circuits

1. Fetch an instruction specified by PC and update the PC
2. Decode the instruction and read operands from registers
3. Execute an arithmetic/logic operation
4. Load/store a value from/to memory if needed
5. Write back the result to a register

- Note: not all instructions require all the above stages, but in hardware we must implement them all

# Classic Five Stages of Processors



# Simple Processor

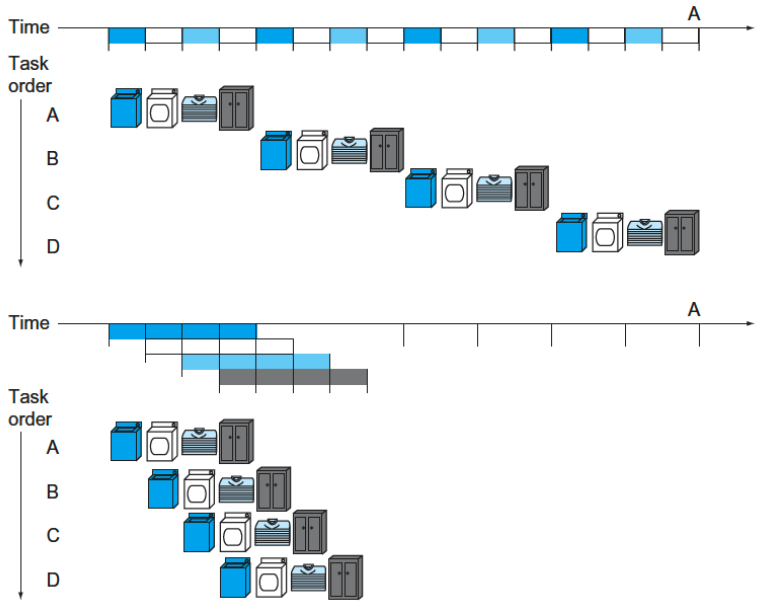
- Each instruction sequentially goes through the 5 stages
  - Skip stages that are not needed, e.g., load/store for arithmetic/logic instructions
  - Time of executing one instruction = sum of all stages
- The next instruction only starts after the current instruction completes
  - Total time of executing a program = sum of all instructions

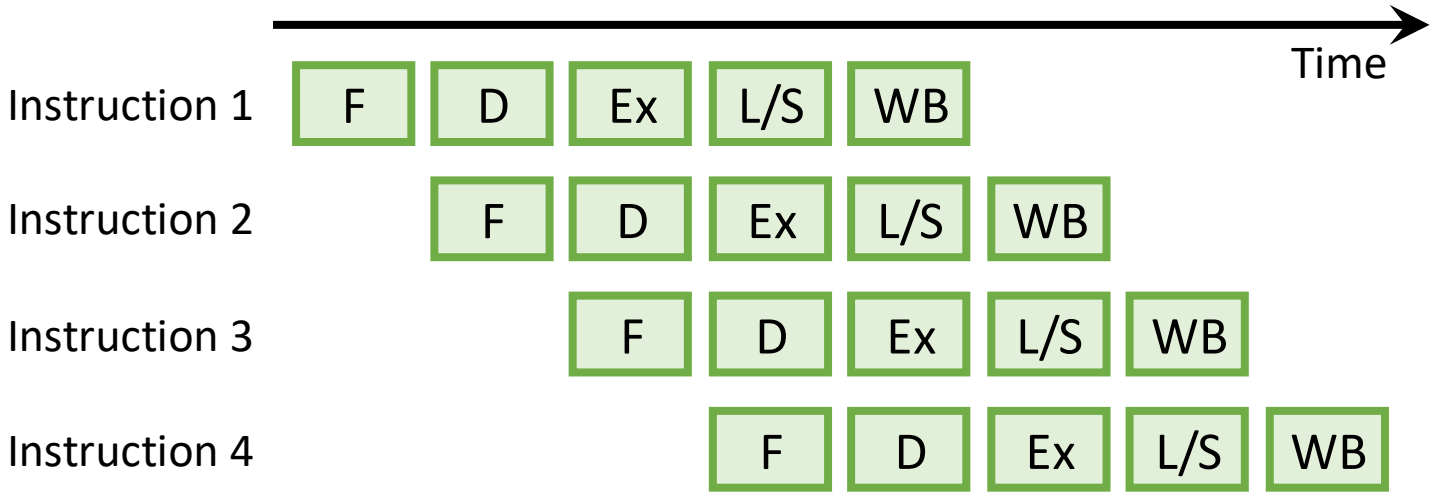
- Simple: a direct realization of the instruction execution model
- Slow: everything is serial; go through each module sequentially

- Again, functionally complete, but performance-wise we can do better

# Optimization 1: Pipelining

- The next instruction starts when the current one goes to the next stage
  - Overlap stages of different instructions
- Pipelining is a form of instruction-level parallelism
  - Multiple instructions simultaneously exist in the processor, but at different stages

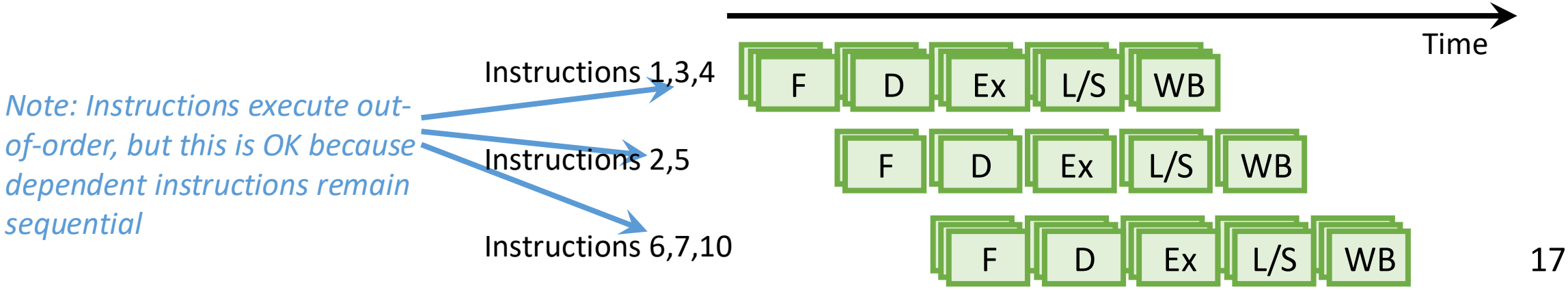


# Optimization 2: Superscalar

- Can we further improve parallelism?
- Many instructions in a program are independent
  - Independent instructions can execute in parallel or out of order
- Hardware contains multiple copies of each stage module, and dynamically find and reorganize ≥ 1 (i.e., superscalar) independent instructions to execute together in each cycle
- Superscalar is also a form of instruction-level parallelism
  - Multiple instructions simultaneously exist in the processor, and at the *same* stage

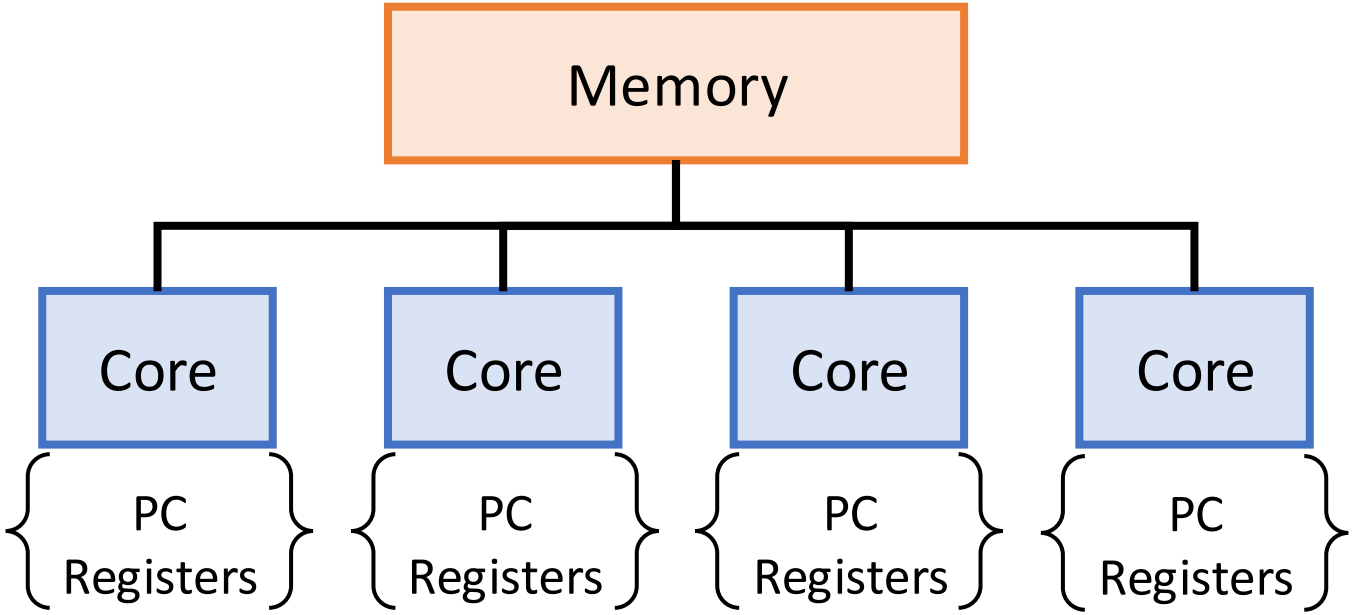
```
x = a + b;
y = c + d;
u = x * y;
```




*Note: Instructions execute out-of-order, but this is OK because dependent instructions remain sequential*

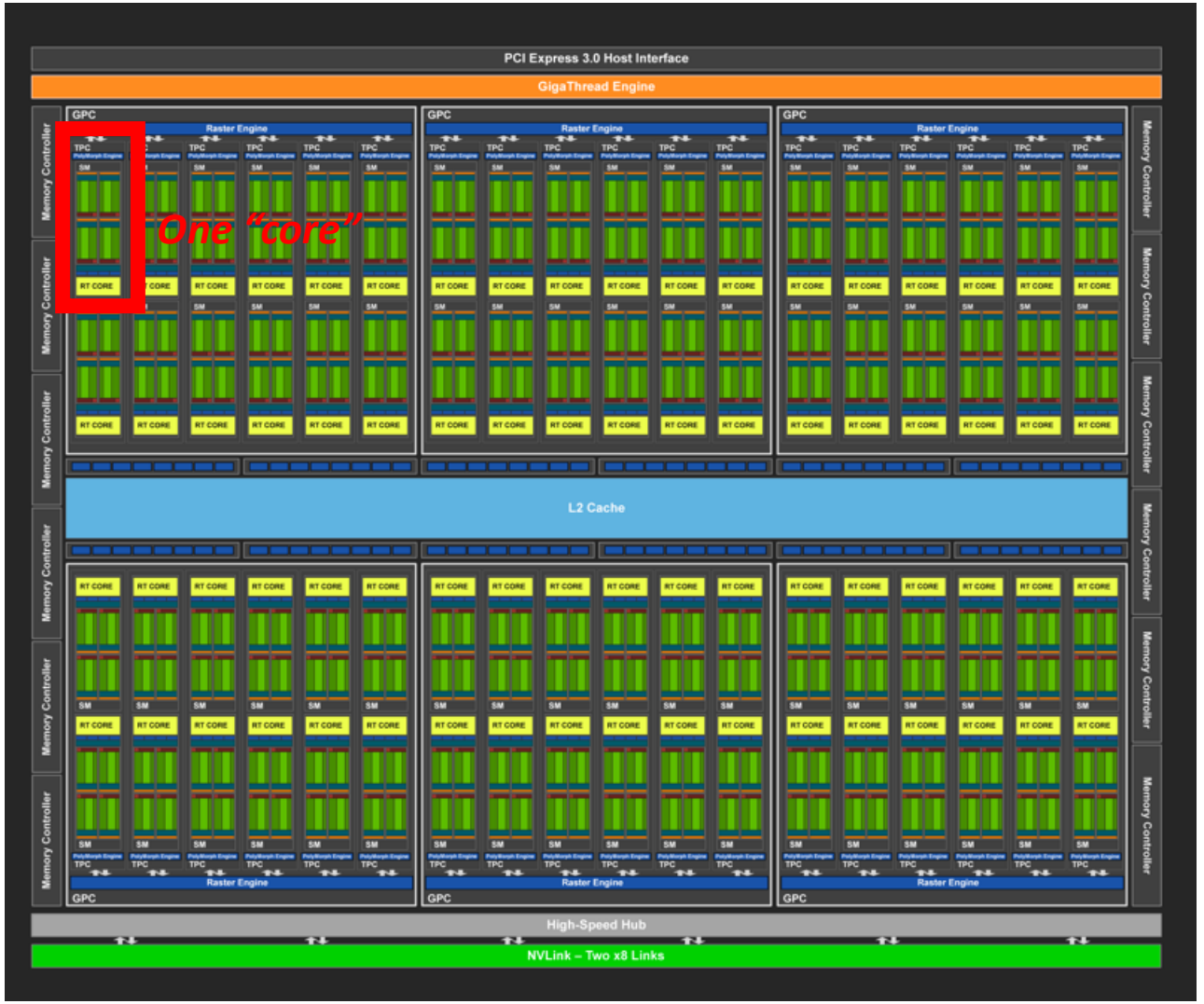
# Optimization 3: Multi-Core

- From a single processor to multiple cores: multiple levels of parallelism
  - PC is private per-core: instruction sequence on each core executes independently
  - Registers are private per-core
  - Memory is shared and accessible to all cores
  - Cores communicate through loads and stores to the shared memory; these accesses are synchronized using special instructions (discussed later)



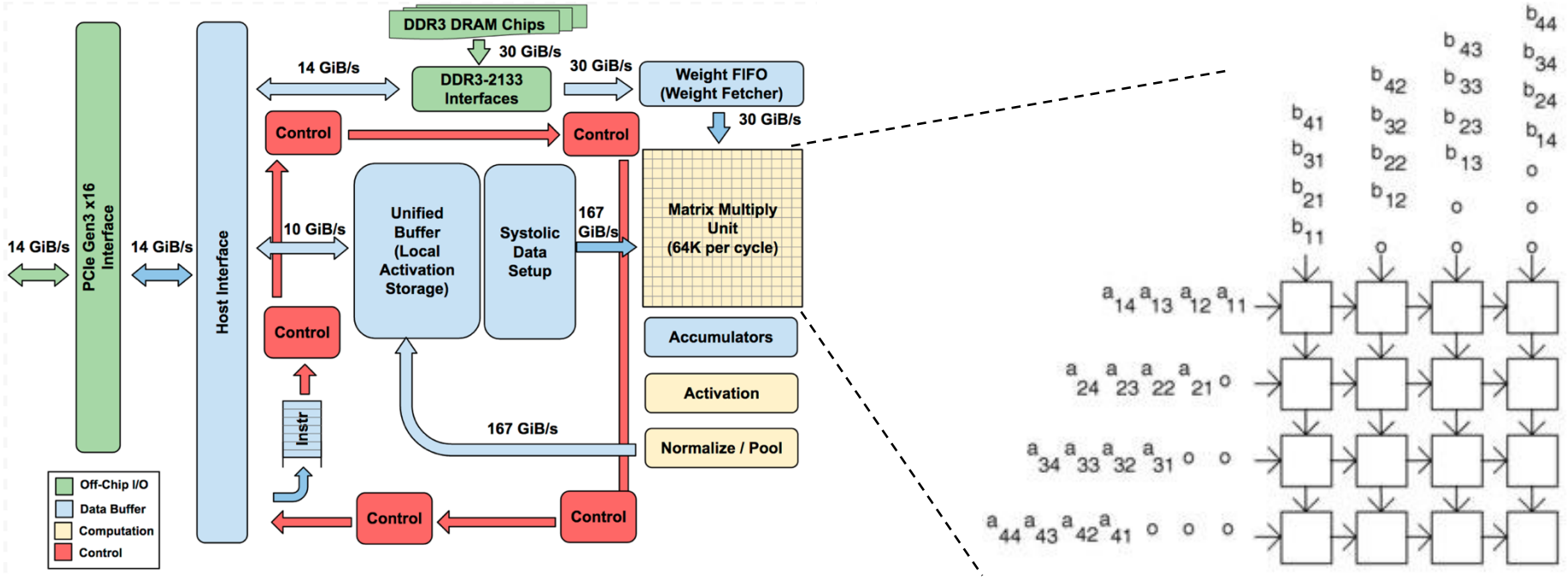
# Example: GPU

- GPU also uses multiple “cores” for a highly parallel processor
- Such parallelism enables very high performance, making it a good fit for compute-intensive graphics and artificial intelligence workloads
- GPUs have other optimizations compared to a typical multi-core CPU core



# Example: Google TPU v1 for Neural Nets

- Matrix Multiply Unit: map $O(n^3)$ ops on a 2D systolic array of multipliers
- On-chip memories: reduce expensive off-chip data transfers
- Interconnection: layer-by-layer processing



# Summary

- Simple and serial processor
- Instruction-level parallelism within one processor
  - Pipelining: overlap
  - Superscalar and OoO: simultaneous
- Parallelism across multiple processors
  - Multi-core
  - GPU

- More hardware details are out of the scope of this course; check out follow-up courses taught by me ☺

# Today's Outline

- High-level models of computers and processors
- Hardware structures of processors
  - Simple, pipelining, superscalar
  - Multi-core and GPU
  - Specialized hardware
- ISA and assembly code
  - System states
  - Arithmetic/logic, load/store, branch/jump instructions
  - Support for parallelism: SIMD and atomic instructions
  - Memory layout of a program: stack management and procedure calls

# Two ISA Styles: RISC vs. CISC

- RISC: reduced instruction set computer
  - Each instruction only specifies a simple operation
  - Fewer instruction types (e.g., 30 – 40) with simple formats in ISA
  - More instructions per program
  - E.g., RISC-V, ARM, MIPS

- CISC: complex instruction set computer
  - Each instruction can specify one or more complex operations
  - More instruction types (e.g., 100 – 200) with complex formats in ISA
  - Fewer instructions per program
  - E.g., x86, x86-64

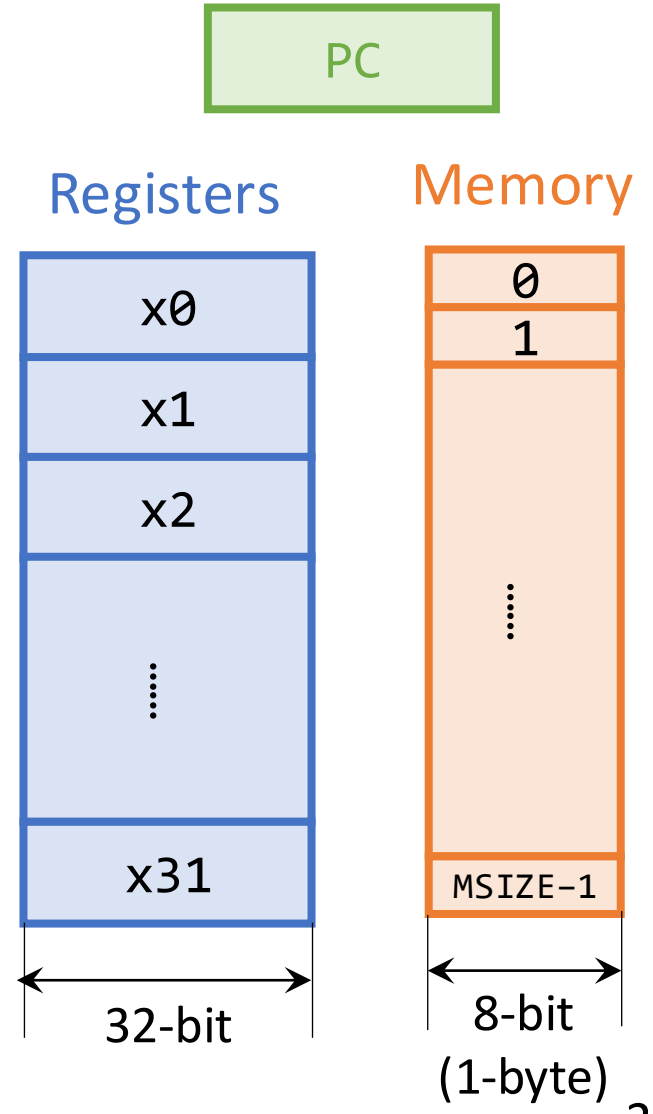
# RISC-V

- The 5th generation of RISC design from UC Berkeley
- A high-quality, license-free, royalty-free RISC ISA specification
  - Very clean and simple ISA
  - Show basic concepts without complicated ad-hoc optimizations

- We talk about the 32-bit of RISC-V basic integer instructions: RV32I
- In this course, you are only required to understand assembly code (assembly → C), not to write assembly code (C → assembly)
  - Instructions appear in lecture notes are required to be memorized
  - Other similar instructions will be explicitly explained if appeared in homework/exams

# RISC-V ISA: States and Instructions

- Program counter (PC): 32-bit
- Registers
  - 32 general-purpose registers (x0 to x31), each 32-bit
  - x0 is always fixed at 0; writing to x0 has no effect
  - There are also other registers, e.g., floating-point
- Memory: byte address
  - Word address is 4-byte aligned, i.e., ending with $00_2$
- All RISC-V instructions are 32-bit long
  - Basic compute: arithmetic/logic/shift/compare
  - Memory access: load/store
  - Control flow: branch/jump



# Arithmetic/Logic Instructions

- `<op>  rd, rs1, rs2`  # inputs are 2 registers
- `<op>i rd, rs1, imm`  # inputs are 1 register and 1 constant
  - Constants are called immediate values in assembly
- E.g., `add  x1, x2, x3; subi x1, x1, 1; xor  x1, x1, x1`
  - Q: What is the effect of the last instruction above?
- Complex operations are broken into a series of simple instructions
  - Recall that RISC-V is a RISC
- E.g., `a = (b ^ c) + d – e;`

```
xor  x28, x2,  x3      # t0 = b ^ c;
add  x29, x28, x4      # t1 = t0 + d;
sub  x1,  x29, x5      # a = t1 – e;
```

  - Q: How many temporary registers are needed *at least* for the above C statement?

# Data Transfer Instructions

- In most RISC ISAs, all memory accesses happen through loads/stores
  - Other instructions only manipulate data within the processor
- Load a 32-bit word (memory to register): `lw rd, offset(rs1)`
- Store a 32-bit word (register to memory): `sw rs2, offset(rs1)`
- Displacement addressing
  - Address is given by a base register and a signed offset immediate
  - Effective address is a byte address = register value + offset
- E.g., `*a = b + c[2]` # assume all are 32-bit `int` value/pointer/array

```
lw   x28, 8(x3)     # c is in x3
add  x28, x2, x28   # b is in x2
sw   x28, 0(x1)     # a is in x1
```

- Q: What are the benefits of displacement addressing?

# Accessing Arrays

- Access an individual array element
  - Store the array base address in the register; use offset value for element index
    - Note: address is in bytes, so multiply the index by the size of the element!
  - E.g., for an int array starting at `0x1000`,
    `&a[0] == 0x1000`, `&a[1] == 0x1004`, `&a[2] == 0x1008`, …
  - E.g., `int x = a[7];` → `lw x28, 28(x1)`
- Use a variable for array index, e.g., iterating
  - Use the base register as the iterator/pointer, and set offset to zero
  - E.g., `a[i] = x; i++;`

```
slli x28, x1,  2      # x28 = 4 * i, byte offset from array base
add  x29, x2,  x28    # x29 = a + 4 * i = &a[i]
sw   x3,  0(x29)
addi x1,  x1,  1
```

  - `slli` instruction: shift logical left by an immediate value

# Changing Program Control Flow

- Normally instructions execute sequentially one after one
- But also need control flow operations, i.e., jump forward/backward
  - E.g., if-else, switch-case, while, for, function/method calls, …

- Two types:
- Conditional branch: based on a condition
  - Mark the target instruction of branch/jump with a label
  - `beq  rs1, rs2, L`  # branch to L if `rs1` == `rs2`
  - Also have `bne`, `blt`, `bge`, …
- Unconditional jump: always change flow
  - `j  L`  # jump to L

# Example of if-else

```
if (a == d)
    a = a + d;
if (i == j)
    a = b + c;
else
    a = b - c;
a++;
```

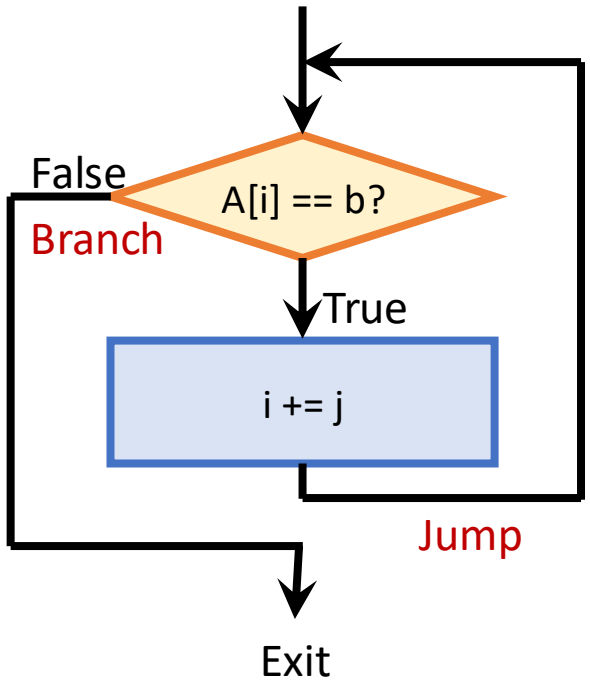
```
# a, b, c, d in x1 to x4
# i, j in x5, x6
      bne  x1, x4, L      # if a != d, goto L
      add  x1, x1, x4     # a = a + d
L:    beq  x5, x6, True   # if i == j, goto True
      sub  x1, x2, x3     # a = b - c, False path
      j    Next           # goto end of branch
True: add  x1, x2, x3     # a = b + c, True path
Next: addi x1, x1, 1      # a++
```

Sometimes we can invert the condition for convenience. *Think carefully about the flow!*

In the above example: `if (a == d)` → bne

# Example of while Loop

```
while (A[i] == b)
    i += j;
```




```
# i in x1, j in x2
# b in x3, A in x4
Loop: slli x28, x1, 2      # t0 = i * 4
      add  x29, x4, x28    # t1 = &(A[i])
      lw   x30, 0(x29)     # t2 = A[i]
      bne  x30, x3, End    # if A[i] != b, go to End
      add  x1, x1, x2      # i += j
      j    Loop            # goto next iteration
End:
```

Can also jump backward!

Q: How about `do-while` and `for` loops?

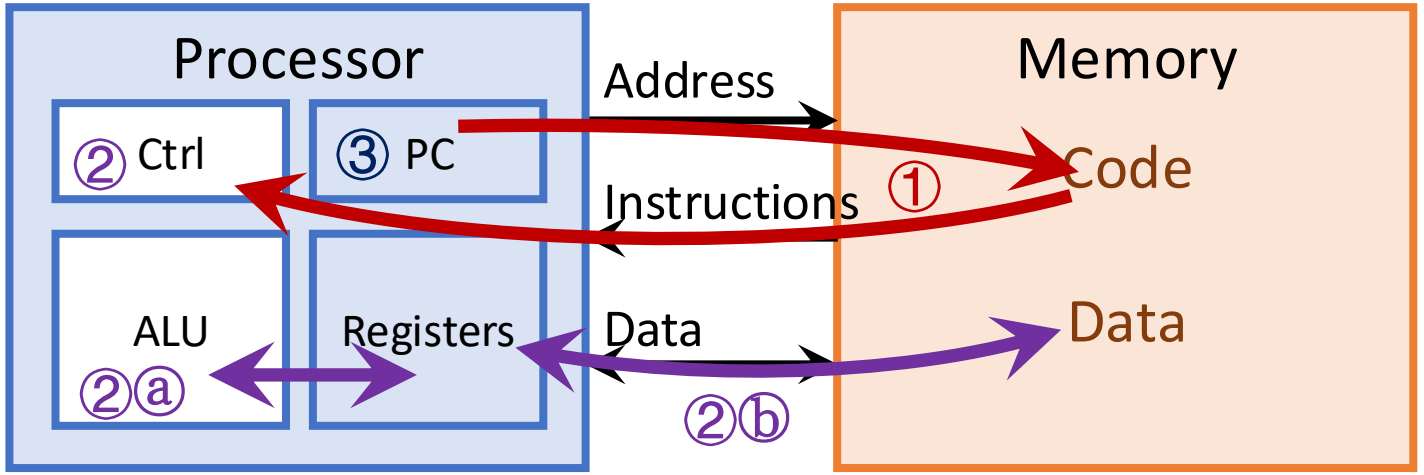
# Review: Instruction Execution on Processors

1) PC used as address to fetch an instruction from memory
2) The controller decodes the instruction and determines what to do
   a) Compute on register values and store results to registers
   b) Transfer data between memory and processor registers
   c) …
3) Update PC to the next instruction, go back to 1)

Arithmetic/logic

Memory load/store

Control flow



# ISA Support for Parallelism

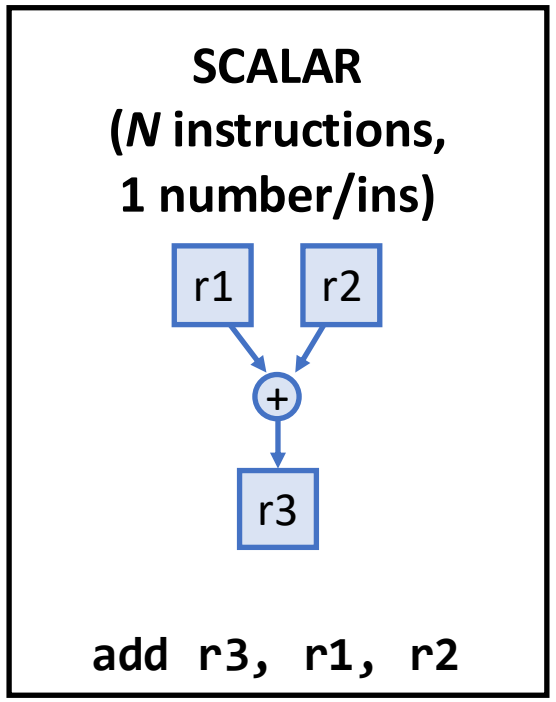
- So far, our ISA is functionally complete on a single processor
- Modern ISAs also feature performance optimizations, e.g., parallelism

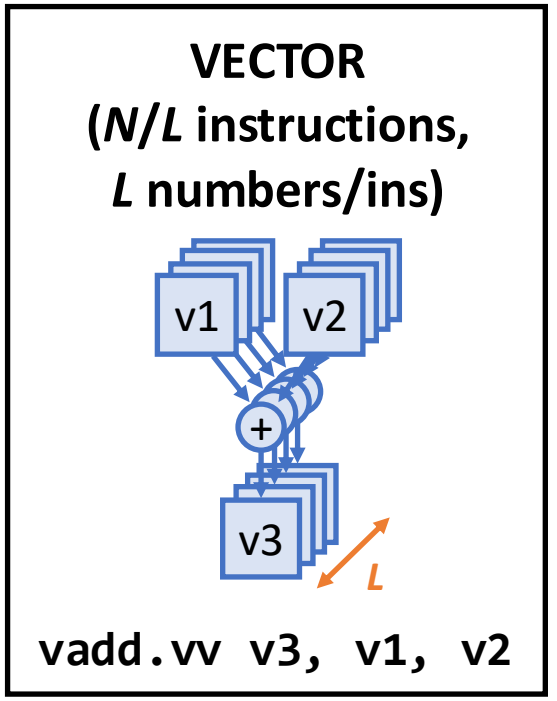
- SIMD (single-instruction, multiple-data)
  - To allow instructions on a processor to process more data simultaneously
- Atomic instructions
  - To coordinate among multiple processors

# SIMD: Single-Instruction, Multiple-Data

- Normal instructions compute on a single number (scalar)
- SIMD instructions compute on a vector of numbers simultaneously
  - Data-level parallelism
  - Examples: Intel SSE, Intel AVX, ARM Neon, …
- Tip: SIMD is useful in lab 1

```
for i: 0 to N
  c[i] = a[i] + b[i]
```



# Case Study: Matrix Multiplication

```
void dgemm(size_t n, double* A, double* B, double* C) {
  for (size_t i = 0; i < n; i++) {
    for (size_t j = 0; j < n; j++) {

      // c0 = C[i,j]
      double c0 = *(C + i * n + j);

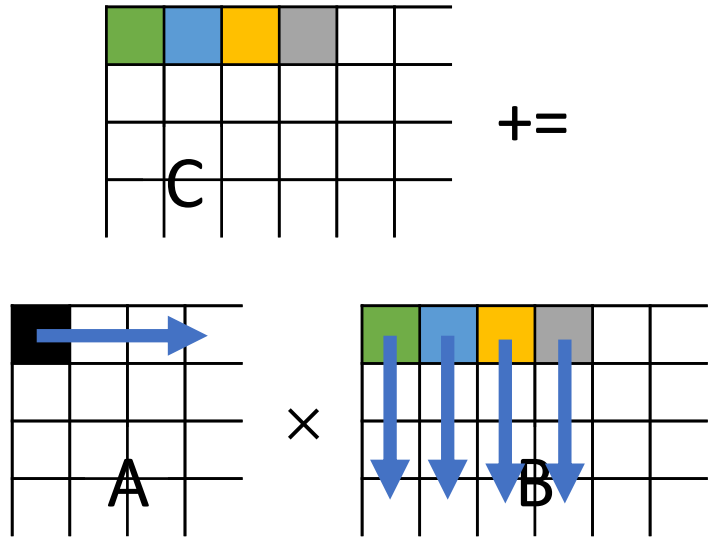

      for (size_t k = 0; k < n; k++) {
        // c0 += A[i,k] * B[k,j]
        c0 = c0 + *(A + i * n + k)) * *(B + k * n + j));
      }



      // C[i,j] = c0
      *(C + i * n + j) = c0;
    }
  }
}
```

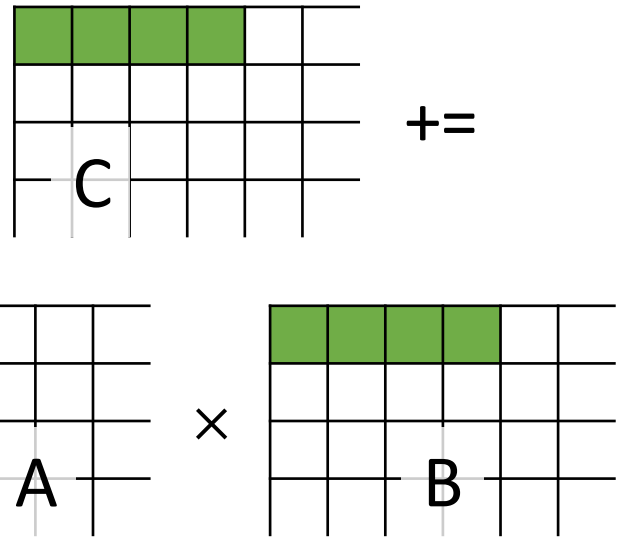
j += 4

*Work on four elements in parallel*



# Case Study: Matrix Multiplication in SSE/AVX

```
void dgemm(size_t n, double* A, double* B, double* C) {
  for (size t i = 0: i < n: i++) {
    for (size_t j = 0; j < n; j += 256 / 8 / sizeof(double)) {
      // c0 = C[i,j]
      // __m256d: a 256-bit register of four doubles
      __m256d c0 = _mm256_load_pd(C + i * n + j);
      for (size_t k = 0; k < n; k++) {
        // c0 += A[i,k] * B[k,j]
        c0 = _mm256_add_pd(
               c0,
               _mm256_mul_pd(
                 _mm256_broadcast_sd(A + i * n + k))
                 _mm256_load_pd(B + k * n + j));
      }
      // C[i,j] = c0
      _mm256_store_pd(C + i * n + j, c0);
    }
  }  // assume n is a multiple of four
}
```




SIMD intrinsics: use C functions instead of assembly to call SSE/AVX instructions

Check the manual to see what intrinsics are available

# Review: Multiple Levels of Parallelism

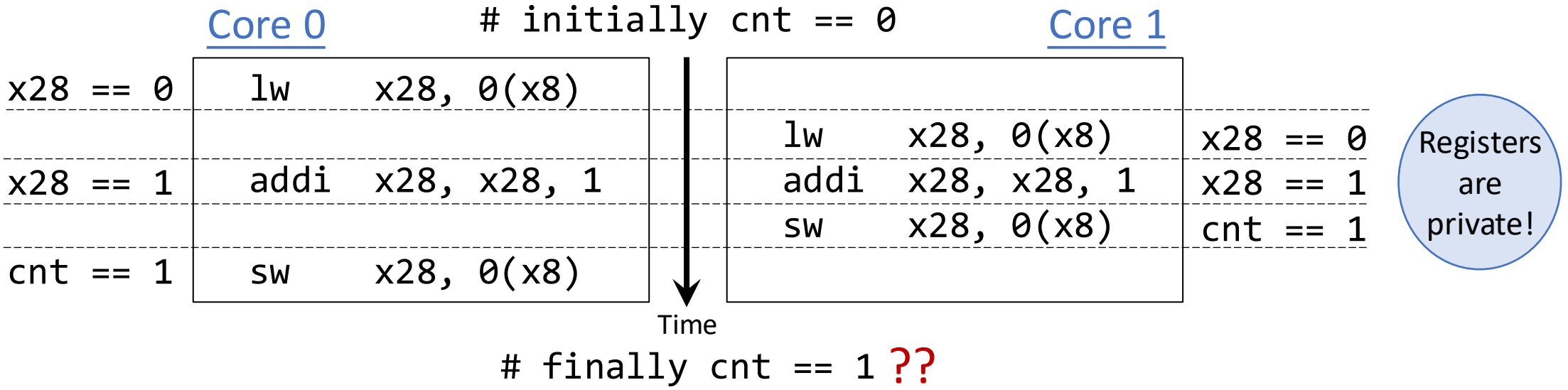
- Not to confuse SIMD with instruction-level parallelism discussed before

- Within a single instruction, process multiple data in parallel
  - SIMD
- Within a single processor, process multiple instructions in parallel
  - Instruction-level parallelism, e.g., pipelining and superscalar
- Among multiple processors
  - Multi-core and GPU

- They can be applied altogether!!
  - E.g., a multi-core CPU uses several superscalar cores, and supports SIMD instructions

# Data Race and Synchronization

- Data race: if multiple cores modify the same data
- E.g., all attempt to increment a shared counter




- Avoid data races by synchronization of loads/stores to shared data
  - Do read-modify-write atomically on shared data
    - Atomicity: as an undividable entity
  - No other access to the data allowed in between

# ISA Support for Synchronization

- Atomic instructions: read-modify-write; many variants in different ISAs
  - Test and set: `t&s reg, addr`
    - `reg = *addr; /* test */  if (reg == 0) *addr = 1; /* set */`
  - Compare and swap
  - Fetch and add/sub
- Use atomic instructions to build locks
  - Use a shared memory location as a lock: 1 means locked; 0 means free
  - Only one core can enter the critical section
  - E.g., use test & set to build a spin lock
    - Q: Think about why can't we replace `t&s` with two normal load and store instructions?

```
lock:   t&s  reg, addr
        bne  reg, zero, lock
               /* if not 0, try again */
```

```
unlock: st   addr, 0
                 /* write 0 to location */
```

  - More about locks in the lectures of parallel programming

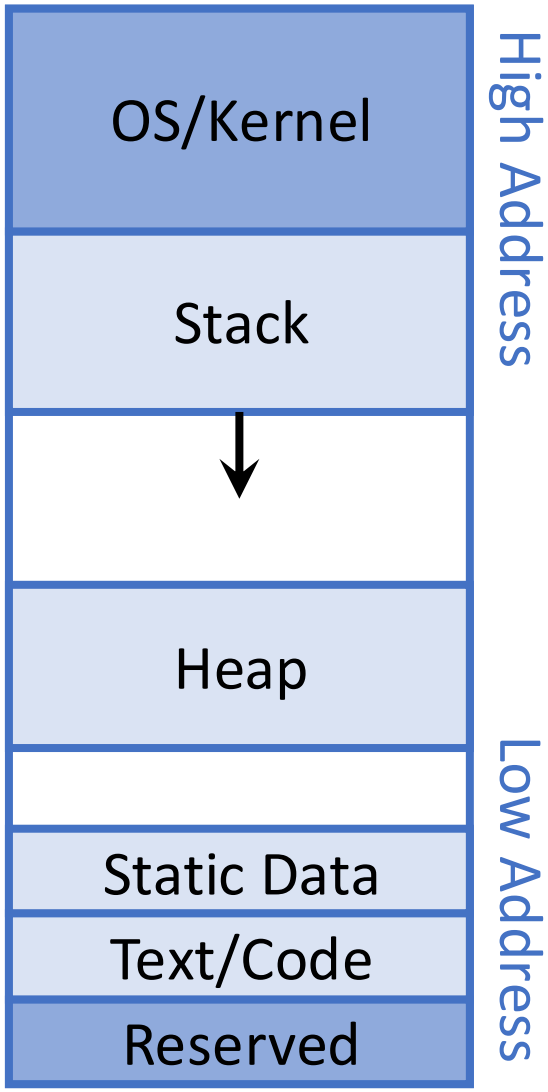
# Today's Outline

- High-level models of computers and processors
- Hardware structures of processors
  - Simple, pipelining, superscalar
  - Multi-core and GPU
  - Specialized hardware
- ISA and assembly code
  - System states
  - Arithmetic/logic, load/store, branch/jump instructions
  - Support for parallelism: SIMD and atomic instructions
  - Memory layout of a program: stack management and procedure calls

# Memory Layout of a Program

- The memory space is divided into different regions
  - Exact mapping layouts vary across OS'es and ISAs

- Code/text, static data, …: set up at program loading
  - These contents are stored in the program binary file
- Heap: dynamic data allocation, managed by OS
  - Heap and dynamic allocation will be discussed in later lectures
- Stack: to support (recursive) procedure calls in hardware
  - Arguments, local variables, temporary variables, …
- OS/kernel, reserved, …



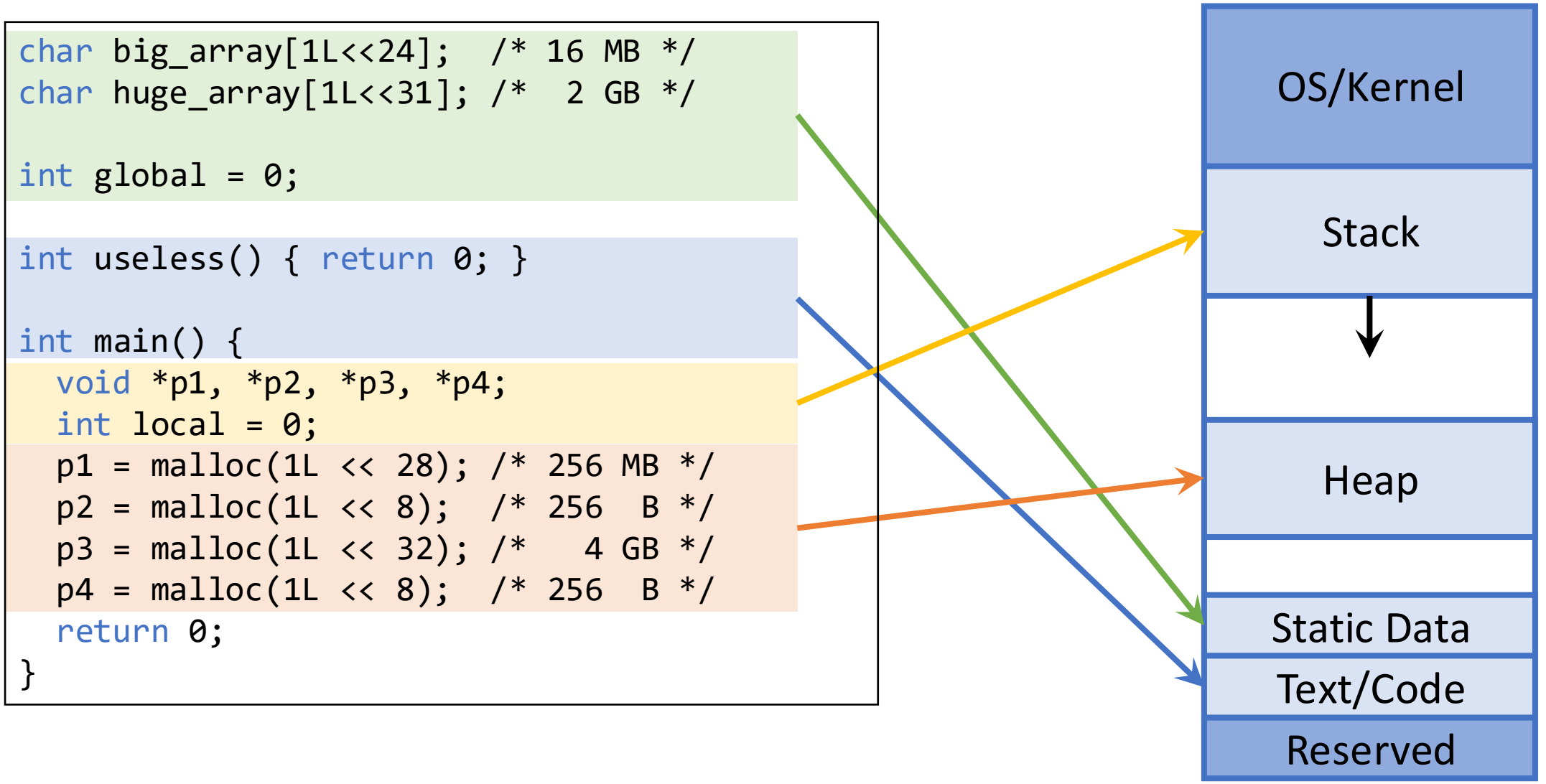
# Memory Layout Example

```
char big_array[1L<<24];  /* 16 MB */
char huge_array[1L<<31]; /*  2 GB */

int global = 0;

int useless() { return 0; }

int main() {
  void *p1, *p2, *p3, *p4;
  int local = 0;
  p1 = malloc(1L << 28); /* 256 MB */
  p2 = malloc(1L << 8);  /* 256  B */
  p3 = malloc(1L << 32); /*   4 GB */
  p4 = malloc(1L << 8);  /* 256  B */
  return 0;
}
```



# Why We Need a Stack

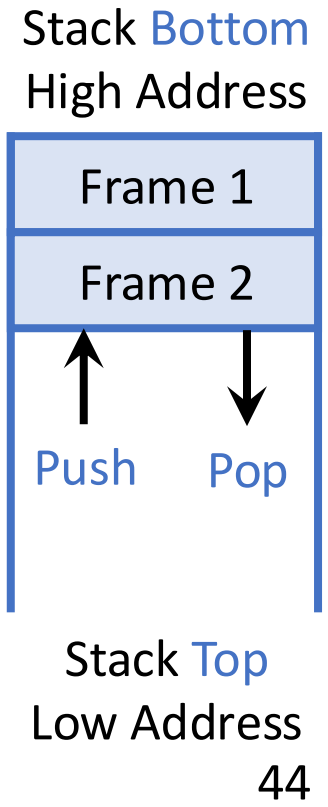
- Many programming languages (e.g., C/C++, Java, …) support procedures
  - A.k.a., functions, methods, subroutines, …
  - Allow better (modular, readable, …) program structures

- A procedure needs memory space for its arguments, local variables, …
  - There could be multiple simultaneous procedure instantiations, e.g., recursion, and each needs a different space
    - E.g., A( ) calls itself; or A( ) calls B( ) calls A( )
  - Procedure call chain is not statically known, so dynamic allocation is needed
    - E.g., `int A(int x) { if (x) return B(); else return C(); }`

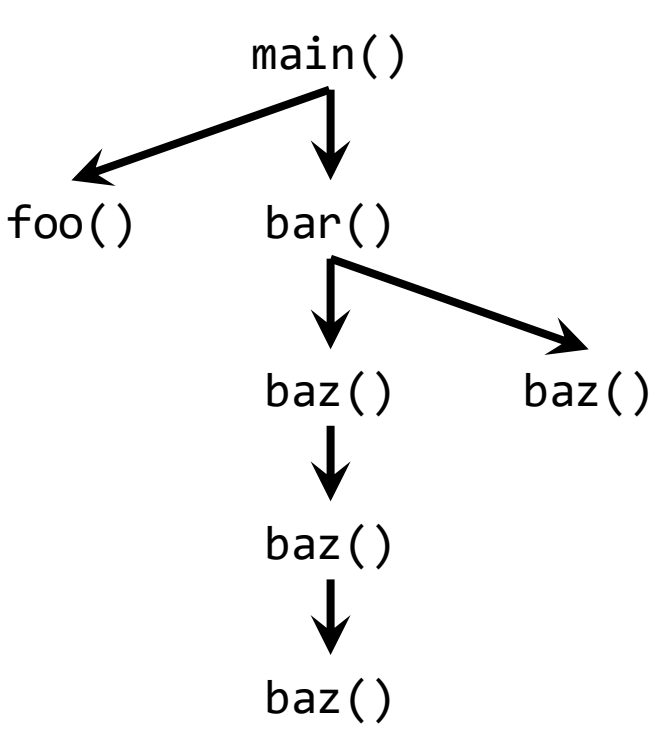
- We could use heap as a general (but slower) solution
- A simpler and more optimized solution exists

# Stack Management

- Observations of procedures
  - Memory space of an instantiation is only needed for limited time, e.g., from when called to when return
  - Procedure calls are usually perfectly nested; callee returns before caller does
- Stack: last in, first out (LIFO), to match nested procedure call
- Frame: the space for each procedure instantiation
  - Allocated when entering procedures; pushed into the stack
  - Deallocated when returning; popped out of the stack
- Stack grows downwards (low address) and shrinks upwards



# Stack Management Example







- Recursion
  - Handled without special consideration
  - Each instantiation gets its own private space, e.g., a stack frame, for local variables
  - Last-in, first-out discipline still applies
- Also works for mutual recursion
  - P calls Q; Q calls P

# Register Usage in Procedure Calls

- Problem
  - When A( ) calls B( )
  - A( ) may use a set of registers, e.g., x1, x2, x3, x4
  - B( ) may also use a set of registers, e.g., x2, x4, x6, x8
  - A( ) and B( ) could be written by different developers (e.g., your code calls a lib function like `printf( )`), so they are not aware of each other
  - Their register usage may overlap! So B( ) corrupts some of A( )'s variables!
  - E.g., after B( ) returns, A( ) may see wrong values in x2 and x4
- Solution
  - Caller (i.e., A( )) or callee (i.e., B( )) would need to save the registers before the call, and restore them after the call
  - Can be done by either caller or callee, and only needed by one side
  - ISA defines for every register, whether caller or callee should save/restore it

# Procedure Calls

- Passing control: jump from caller to callee; return back
  - `jal L`: jump-and-link, jump to L and save return position to register `ra` (x1)
  - `jr ra`: jump to the address stored in `ra`
- Passing data: procedure arguments and return values
  - Use registers to store arguments and return values
  - If there are too many arguments, put on the stack frame
- Local/temporary variables
  - Allocated on the stack frame
- Save/restore registers
  - On the stack frame

# Next Lecture:
# Memory Hierarchy

# Appendix

# Key Idea: Abstraction through Leveled Interface

- (High-level) programming languages, e.g., C/C++
  - Programmers only need to understand the high-level language specifications
  - Compilers ensure that the programs can run on different ISAs
- ISA (i.e., processor architecture)
  - The parts of a processor that one (or the compiler) needs to understand to write (assembly/machine) code
  - Do not expose hardware details; processors evolve with technology *without* changing the interface
    - E.g., Intel Pentium 2, 3, and 4, M, Core, Core 2, … all use x86
- Microarchitecture: hardware implementation details
  - Do not have to know to write *correct* code, but must know to write *efficient* code
  - E.g., cache sizes, core frequency, pipeline depth, …

# Specialized/Customized Hardware

- High flexibility: processors
  - Flexible: can implement any algorithm with a generic set of instructions
  - Inefficient: instruction and control overheads, i.e., programmability overheads
- High efficiency: specialized hardware, a.k.a., accelerators
  - Efficient: only build the hardware you need, no programmability overheads
  - Inflexible: can only compute one algorithm or one application domain
    - Specific mix of math and control units, specific data paths, specific precision, …
- Designing a good specialized hardware accelerator usually requires good understanding of the corresponding domain
  - E.g., AI, crypto, database, genomics, …

# CISC vs. RISC: Historical Debate

- Early processors were CISC
  - Memory was small: each program can only contain a few instructions
  - Programmers worked in assembly; would like to use more expressive instructions
- RISC emerged around 1980
  - Simpler instructions are easier for compilers of high-level languages
  - Processor design complexity increases: designing RISC processors are easier
  - Energy and power become primary constraints: decoding CISC has high overhead
- The line between RISC and CISC processors is blurred these days
  - CISC (e.g., x86) remains dominant now due to legacy and commercial interests
  - CISC processors internally translate code into RISC-style micro-ops to execute
  - Q: Does the ISA choice still matter?

# ISA Support for Synchronization

- Atomic instructions: read-modify-write; many variants in different ISAs
  - Test and set: `t&s  reg, addr`
    - `reg = *addr; /* test */  if (reg == 0) *addr = 1; /* set */`
  - Compare and swap: `cas  cmp_reg, swap_reg, addr`
    - `if (*addr == cmp_reg) swap(*addr, swap_reg);`
  - Fetch and add/sub
    - Read from memory; apply operation on it and store back; return the original value
    - RISC-V, A-extension: fetch and swap/add/and/or/xor/max/min

      ```
      AMOSWAP  rd, rs2, (rs1)
      AMOADD   rd, rs2, (rs1)
      ...
      ```

      Read data from address `rs1`; apply the operation with `rs2` and store back; leave the original value in `rd`